



마이보이스 (유저앱 + 덤라이트 제작 파이프라인) 더빙 체험 기능 개발

1. 프로젝트 개요

- **프로젝트명:** 마이보이스 더빙 체험 기능 개발 (이용자 앱 + 덤라이트 제작 파이프라인)
- **개발 기간:** 2026.05 ~ 2026.06
- **플랫폼/환경:** React + Next.js + TypeScript (이용자 앱) / 덤라이트 검수(백오피스) 프론트
- **담당 범위**
 - 스트리밍형 더빙 체험 기능 기술 PoC 및 프론트 기술 검토
 - 마이보이스 웹/모바일 녹음 플로우 구현
 - 마이보이스 플레이어 API 연동 및 독립 플레이어/탭 구조 개편
 - 덤라이트(검수) 기반 마이보이스 설정 CRUD 및 구간 정의 로직
 - 구간 추출 사이드바 / 크롭 확대 뷰어 / 좌표 태깅 UI 구현
 - content-hub API 연동(썸·좌표 생성·조회·삭제, presigned URL S3 업로드)
 - 픽셀 크롭 및 비율 좌표 저장, 주요 캐릭터 지정 + QA

2. 요구사항 및 나의 역할

요구사항 요약

- 스트리밍형 더빙 체험은 구현 전 실패 가능성이 높은 영역(동기화/네트워크/저장/보안)을 먼저 검증해야 했음
- "제작 → 노출 → 이용"으로 이어지는 전체 파이프라인이 단일 흐름으로 연결되어야 했음
- 연출용 키프레임(spoint)이 대사 위치의 절대 기준이 되기 어려운 구조적 한계를 해소해야 했음

- 관리자 ↔ 이용자 렌더 스케일 차이에서도 말풍선 좌표가 어긋나지 않아야 했음

담당 역할

- **스트리밍 더빙 PoC/기술 검토:** 동기화·네트워크·저장·보안 URL·progressive MP4 등 프론트 관점 리스크를 PoC 항목으로 정의
- **녹음 플로우/플레이어 구현:** 웹/모바일 녹음 플로우와 플레이어 API 연동, 독립 플레이어/탭 구조 개편
- **딥라이트 제작 측 구현:** 마이보이스 설정 CRUD, 마커 기반 구간 정의/겹침 검사 로직
- **좌표 태깅/크롭/연동:** 좌우 분할 줌 뷰어 좌표 태깅, 픽셀 크롭, content-hub API 연동
- **캐릭터 지정/QA:** 주요 캐릭터 지정 기능 및 전반 검증

3. 기술 스택 및 아키텍처

항목	사용 기술 / 접근 방식
프레임워크	React, Next.js, TypeScript
미디어/녹음	웹/모바일 녹음 플로우, progressive MP4, 보안 URL/프리로드
저장/연동	content-hub API, presigned URL 기반 S3 업로드, buildOnly/onApply 분리
좌표/크롭	마커(ms) 기준 대사 수집 + 픽셀(파란 줄) 기준 크롭, 절대 px + 비율(%) 이중 저장

아키텍처 특징

- “제작(딥라이트) → 노출(관리자) → 이용(이용자 앱)”을 단일 파이프라인으로 설계
- 연출/대사 타임라인 어긋남을 시간(마커)과 픽셀(이미지) 이중 기준으로 분리해 정합성 확보
- 직접 좌표 태깅을 우선 사용하되 기존 추정 매핑을 fallback으로 유지하는 이중 안전 구조

4. 주요 기능

기능	설명
스트리밍 더빙 PoC	동기화/네트워크/저장/보안 URL/progressive MP4 검토
웹/모바일 녹음 플로우	권한·안내·상태 전이·예외 케이스 분리 구현
독립 플레이어/탭 구조	라우팅/레이아웃/상호작용 분리로 결합도 감소

기능	설명
마이보이스 설정 CRUD	생성/수정/삭제/조회 + 플레이어 연동
구간 추출 사이드바	모달 → seamless 사이드바, 칩형 요약, raw 세트 데이터
좌표 태깅	좌우 분할 줌 뷰어, 대사 순서 핀 태깅(% → px → 이미지 분기)
content-hub 연동	썸·좌표 CRUD, presigned URL S3 업로드, buildOnly/onApply 분리
픽셀 크롭/비율 좌표	파란 줄 기반 크롭, 절대 px + 비율(%) + 썸 크기 저장
주요 캐릭터 지정	캐릭터 목록·썸네일·is-main 저장/잠금/복원

5. 주요 성과

- **“제작 → 노출 → 이용” 전체 파이프라인 구축:** 검수 페이지에서 녹음 구간·크롭 이미지·말풍선 좌표·주요 캐릭터를 생성하면 content-hub 저장 → 관리자 작품 생성 → 이용자 대사 선택/녹음으로 이어지는 단일 흐름 설계·구현
- **연출/대사 타임라인 어긋남 해소:** spoint가 절대 기준이 되기 어려운 한계를 시간 기준 대사 수집과 픽셀 기준 이미지 크롭으로 분리해 정합성 확보
- **추정 매핑을 직접 좌표 태깅 워크플로우로 전환:** 줌 뷰어에서 대사 순서대로 말풍선을 페어링하고, 절대 px + 비율(%) 이중 저장으로 렌더 스케일 차이에서도 위치 안정화
- **스트리밍 더빙 체험 기술 리스크를 PoC 기준으로 선제 정의:** 재생/녹음/동기화/네트워크/저장/보안 영역을 체크리스트화해 이후 의사결정 속도 향상
- **녹음 플로우·독립 플레이어 구조를 실사용 가능한 형태로 구현:** 권한·안내·상태 전이·예외 케이스를 분리하고 독립 구조로 개편해 확장 시 결합도 감소

6. 문제 해결 및 기술적 도전

✓ 문제 1: 스트리밍 더빙 체험의 기술 리스크를 구현 전에 특정해야 했던 문제

‘바로 구현’보다 실패할 수 있는 지점을 먼저 정의하는 것이 중요했음. 동기화 정밀도, 네트워크 지연/재시도, 저장/업로드 안정성, 보안 URL/프리로드, progressive MP4 운영 방식을 PoC 항목으로 정리해 이후 개발 단계의 의사결정 속도를 높였음.

✓ 문제 2: spoint가 대사 위치의 절대 기준이 되기 어려운 구조적 한계

spoint는 화면 연출용 키프레임이라 대사 위치의 절대 기준으로 쓰기 어려웠음. 이를 시간(ms) 기준 대사 수집과 픽셀(파란 줄) 기준 이미지 크롭으로 분리하는 “마커+픽셀” 이중 기준으로 정합성을 확보했음.

✓ 문제 3: 추정 기반 매핑의 한계를 직접 좌표 태깅으로 전환해야 했던 문제

추정 매핑은 이동/썸네일 보정 규칙이 어긋나면 동기화가 깨졌음. 크롭 이미지 줌 뷰어에서 대사 순서대로 말풍선을 직접 페어링하는 좌표 태깅 워크플로우를 도입해 신뢰도를 높였음.

✓ 문제 4: 관리자 ↔ 이용자 렌더 스케일 차이에서도 위치를 보장해야 했던 문제

좌표를 절대 px + 비율(%) + 씬 크기(xPixel/yPixel)로 함께 저장하고 다썸 누적 Y 계산·하위 호환을 처리해, 서로 다른 스케일에서도 동일 상대 위치를 복원할 수 있게 했음.

✓ 문제 5: content-hub 재진입 시 중복 씬 생성 문제

썸·좌표 생성/조회/삭제와 presigned URL S3 업로드를 연동하면서, buildOnly/onApply를 분리해 재진입 시 중복 씬이 생성되지 않도록 했음.

7. 협업 및 커뮤니케이션

- 제작(덱라이트)·관리자·이용자 측 데이터 흐름을 단일 파이프라인 관점에서 정리해 팀 간 인터페이스 합의를 명확히 함
- PoC 항목을 체크리스트로 공유해 구현 난이도/리스크에 대한 사전 합의 비용을 낮춤
- 좌표/크롭 데이터 포맷(절대 px + 비율)을 표준화해 백오피스 ↔ 이용자 앱 간 해석 차이를 줄임

8. 개발 결과 및 회고

결과

- 검수 → 관리자 → 이용자로 이어지는 마이보이스 콘텐츠 제작·연동 파이프라인을 프론트 관점에서 구축
- 웹/모바일 녹음 플로우와 독립 플레이어 구조를 실사용 가능한 형태로 정리

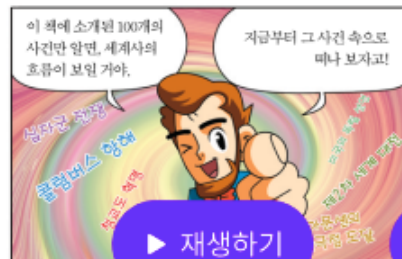
회고

- 미디어 동기화 문제는 단일 기준(시간 또는 픽셀)만으로는 한계가 있었고, 시간과 픽셀을 분리한 이중 기준 + 직접 태깅/추정 fallback 구조가 적합성과 안정성 모두에 효과적이었음

9. 스크린샷

원본듣기

마이 보이스



▶ 재생하기



이미지 1. 마이보이스 메인 화면



🔊 원본 듣기

1

2

3

🔊 하나씩 녹음하기

🔊 썸으로 녹음하기

⊗ 마이보이스 취소

이미지 2. 마이보이스 녹음 선택 화면



2 하지만 세계 각국은 너무나 복잡해요.

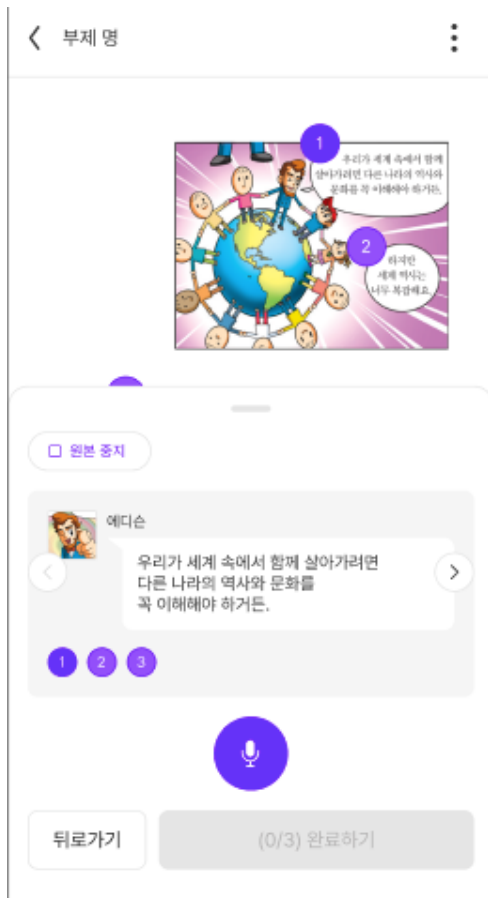


완료 0/3

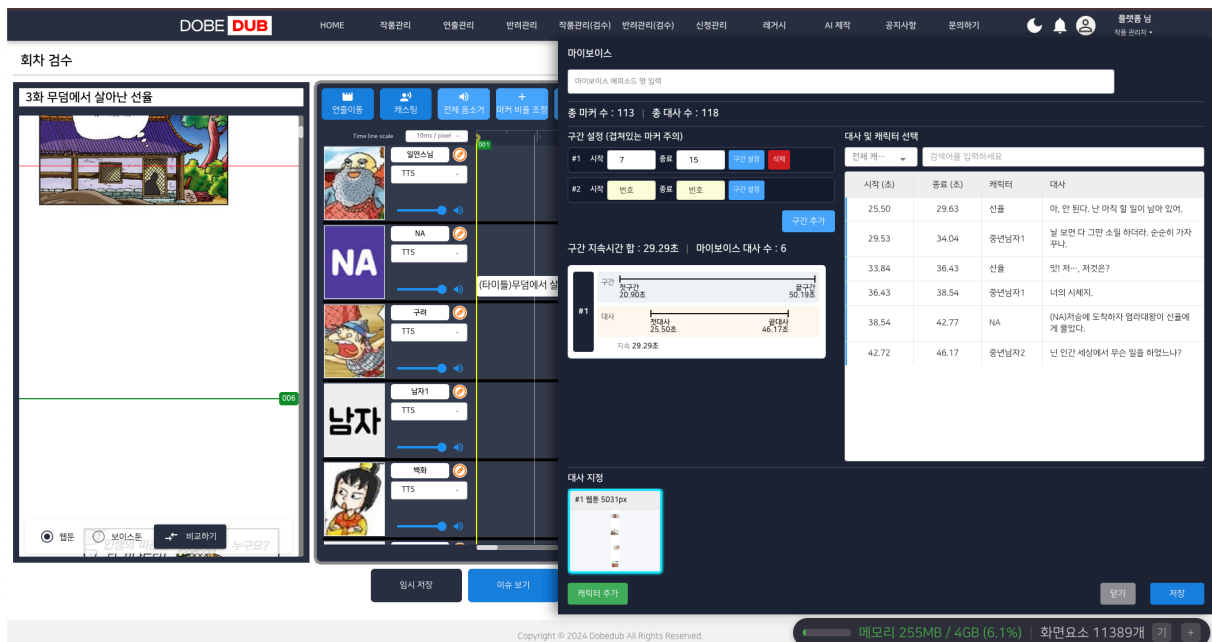
00

00 : 00.4
00.0초





이미지 3. 마이보이스 녹음/확인 화면



이미지 4. 덤라이트 마이보이스 제작 화면

·
·